

Tecnológico de Monterrey

Evidencia 2
Actividad Integral de Estructuras de Datos Lineales

Pamela Hernández Camacho | A01771362

Profesor: David Alonso Cantú Delgado
18 de Octubre, 2026
Av. Eugenio Garza
Sada 2501 Sur, Tecnológico
64700 Monterrey, N.L.

Programación de estructuras de datos y algoritmos fundamentales
(Grupo 607)

1. Objetivo, alcance y entorno

Se desarrolló una aplicación de C++17 que procesa registros de incidentes de red. Las líneas se reciben en una Fila (FIFO), se validan y se guardan en una lista doblemente ligada. Un Merge Sort estable ordena los nodos cronológicamente; luego se permiten búsquedas por día, visualización descendente mediante una Pila (LIFO) y estadísticas con empates.

Entorno probado: macOS, compilador g++, ejecución desde Evidences/Evidence2. Se utilizaron dos archivos reales de 6,818 líneas cada uno: data/log607-1.txt (aleatorio) y data/log607-2.txt (casi ordenado).

2. Estructuras de datos y funciones

Estructura	Función	Motivo
Lista ligada simple	Base para Fila y Pila	Inserción y extracción por extremos sin std::queue ni std::stack
Fila (Queue)	Lectura de líneas	Conserva el orden de llegada para la validación
Lista doblemente ligada	Almacén de registros y ordenamiento	Permite enlaces prev/next y reordenamiento directo
Pila (Stack)	Listado descendente por día	Invierte el orden tras una búsqueda ascendente

3. Evidencia de ejecución: Parte A

La salida guardada en docs/pruebas_estructuras.txt comprueba las operaciones de lista vacía, un elemento, inserción/eliminación por extremos, LIFO, FIFO, recorrido bidireccional e integridad de enlaces tras Merge Sort.

```
Lista vacia: no se puede eliminar.
Pila LIFO: 30 20 10
Fila FIFO: 10 20 30
Lista doble adelante: 1 2 3 4
Lista doble atras: 4 3 2 1
Merge Sort nodos: 1 2 2 3 4
Todas las pruebas terminaron correctamente.
```

Figura 1.

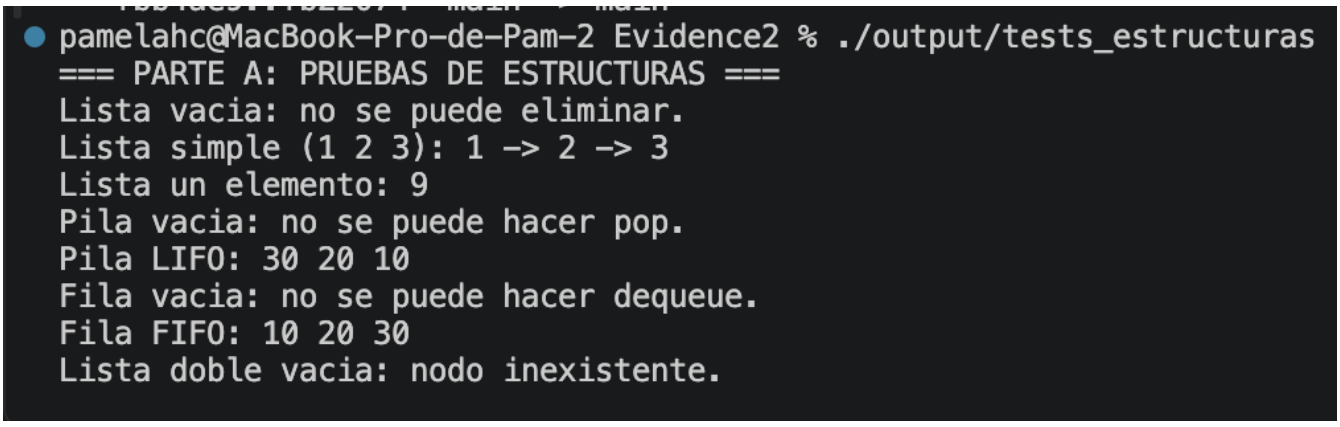
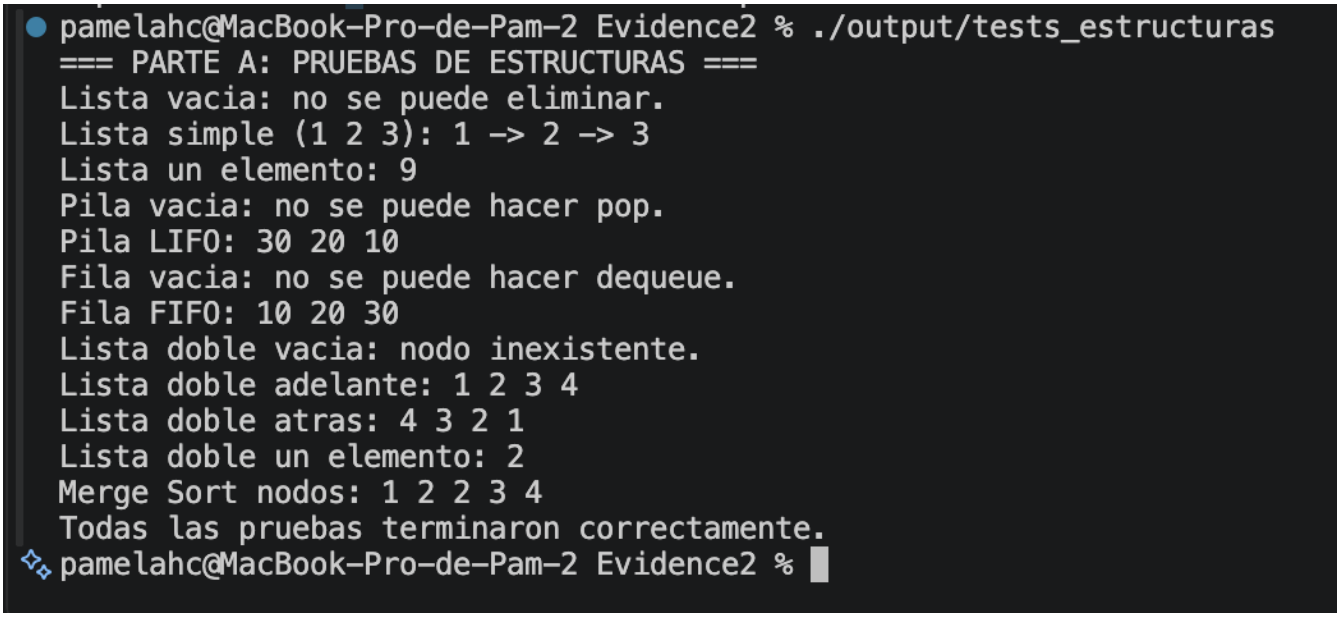


Figura 2.

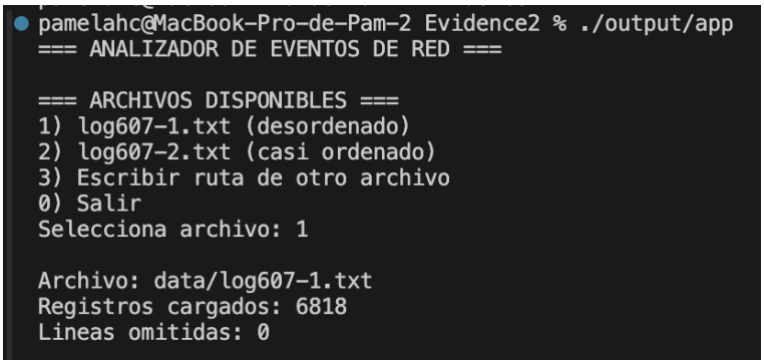


4. Evidencia de ejecución: aplicación y menú

Para el primer archivo, el programa cargó 6,818 registros y omitió 0 líneas. Antes del ordenamiento, la opción de buscar por día notificó que primero se debía usar la opción 1. Al consultar Sep 14 2024 se encontraron 12 registros, y la opción 3 mostró los mismos 12 en orden inverso mediante Pila.

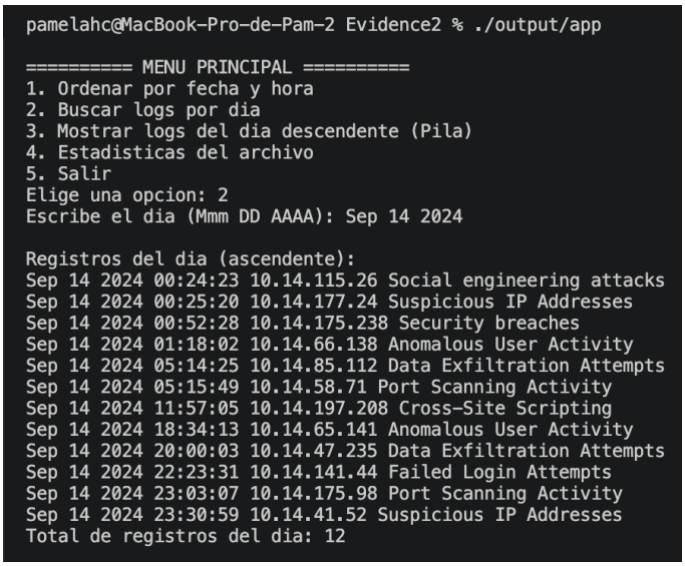
```
Archivo: data/log607-1.txt
Registros cargados: 6818
Lineas omitidas: 0
Primero debes ordenar la lista con la opcion 1.
Total de registros del dia: 12
La lista ya esta ordenada; no se ejecuta Merge Sort otra vez.
```

Figura 3. ***Carga de registros***



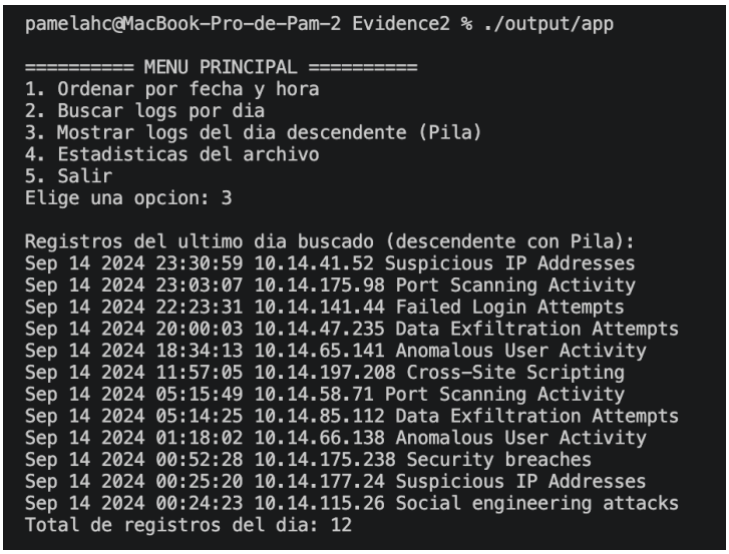
Selección de log607-1.txt, 6,818 registros cargados y cero líneas omitidas. Fuente: captura de terminal proporcionada para la evidencia.

Figura 4. **Búsqueda ascendente**



Consulta Sep 14 2024: se encontraron 12 registros ordenados de menor a mayor hora. Fuente: captura de terminal proporcionada para la evidencia.

Figura 5. ***Salida descendente con Pila***



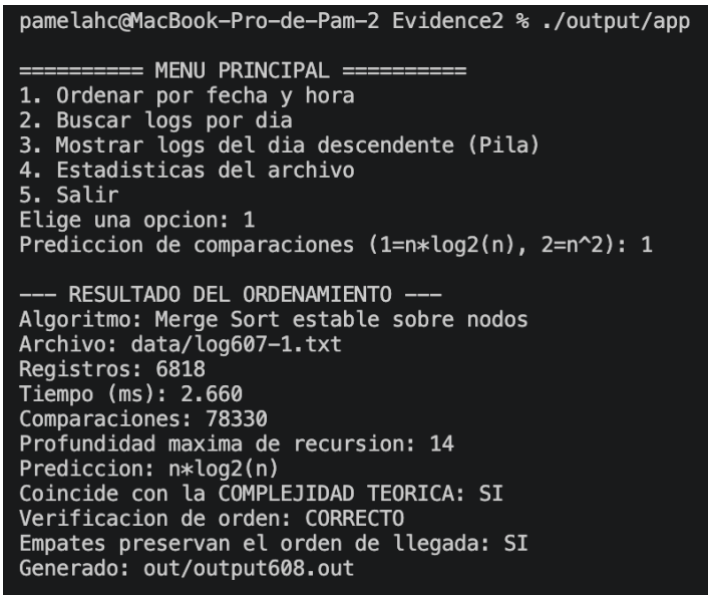
Los mismos 12 registros se muestran en orden inverso con la estructura LIFO. Fuente: captura de terminal proporcionada para la evidencia.

5. Resultados del ordenamiento

Indicador	**log607-1 (aleatorio)**	**log607-2 (casi ordenado)**
Registros cargados	6,818	6,818
Predicción	n · log2(n)	n · log2(n)
Tiempo observado	3.196 ms	0.531 ms
Comparaciones	78,330	50,624
Profundidad máxima	14	14
Orden cronológico correcto	Sí	Sí
Empates estables	Sí	Sí

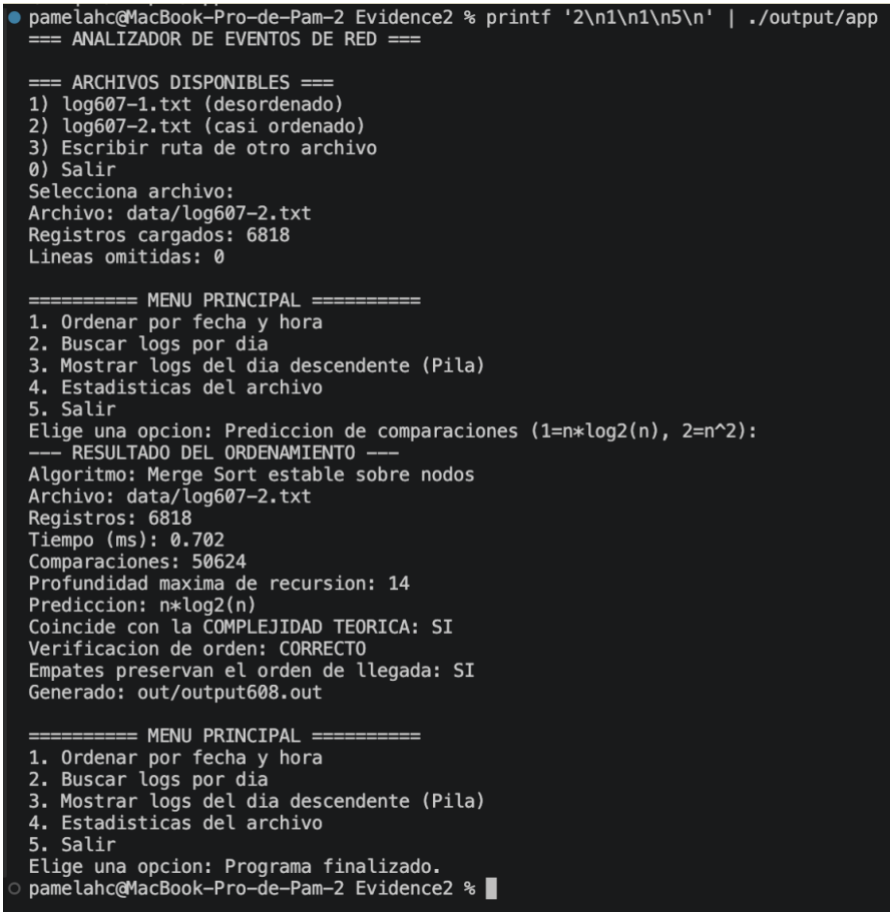
Interpretación: con el mismo número de registros, la lista casi ordenada necesitó menos comparaciones (50,624 frente a 78,330). Merge Sort mantiene complejidad teórica O(n log n); en estas ejecuciones conservó el orden de llegada de registros con fecha y hora idénticas. Ambos recorridos alcanzaron profundidad máxima 14. Los tiempos dependen de la máquina y de la carga del sistema: otra ejecución del segundo archivo midió 0.538 ms y otra 0.760 ms.

Figura 6 ***Ordenamiento cronológico***



Merge Sort estable: 78,330 comparaciones, profundidad máxima 14 y resultado correcto. El tiempo de esta captura es 2.660 ms (otra corrida real distinta a la tabla comparativa). Fuente: captura de terminal proporcionada para la evidencia.

Figura 7. ***Ordenamiento del segundo log y sus mediciones***



Se observaron 6,818 registros, 50,624 comparaciones, profundidad 14 y preservacion del orden de llegada. Esta captura registra 0.702 ms; otras ejecuciones pueden variar.

6. Casos de borde y validaciones

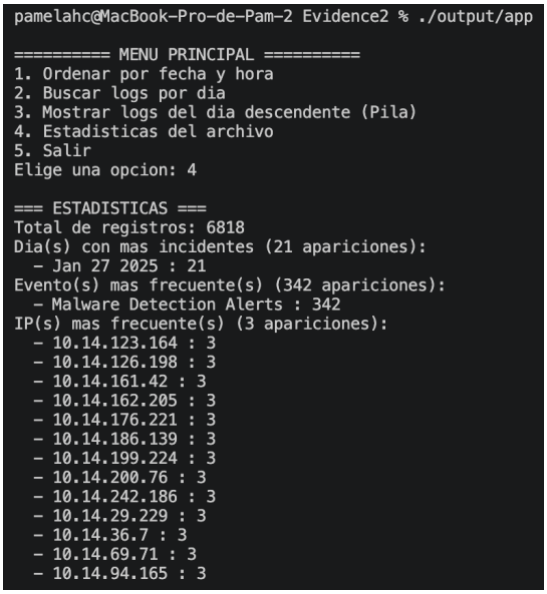
Se creó tests/log_casos_borde.txt con cuatro registros válidos y cuatro inválidos. El programa omitió las líneas 5 a 8 con mensajes distintos y continuó sin cerrarse.

Caso	**Resultado observado**
Día inexistente, Sep 31	Línea omitida; fecha fuera de rango
Hora inválida, 25:00:00	Línea omitida; hora inválida
IPv4 inválida, 999.14.1.3	Línea omitida; IP inválida
Descripción vacía	Línea omitida; falta descripción
Buscar antes de ordenar	Mensaje indicativo; no ejecuta búsqueda
Búsqueda Sep 99 2024	Fecha inválida
Búsqueda Jan 01 2023	0 resultados y mensaje explicativo
Ordenar por segunda vez	Detecta lista ordenada; no repite Merge Sort
Archivo inexistente	Muestra error y regresa al selector
Empates estadísticos	Muestra ambas fechas, eventos e IP empatadas

```
Archivo: tests/log_casos_borde.txt
Registros cargados: 4
Lineas omitidas: 4
Comparaciones: 5
Profundidad maxima de recursion: 3
Verificacion de orden: CORRECTO
Empates preservan el orden de llegada: SI
```

En la prueba de cuatro registros, Sep 14 y Sep 15 empataron con dos incidentes; también hubo empate de eventos y de IP con dos apariciones cada uno. Los archivos de salida de estas pruebas están guardados en docs/.

Figura 8. *Estadísticas generales*



6,818 registros; día con más incidentes Jan 27 2025 (21); evento más frecuente Malware Detection Alerts (342); empates de IP con 3 apariciones. Fuente: captura de terminal proporcionada para la evidencia.

Figura 9. ***Cuatro errores detectados y cuatro lineas aceptadas***

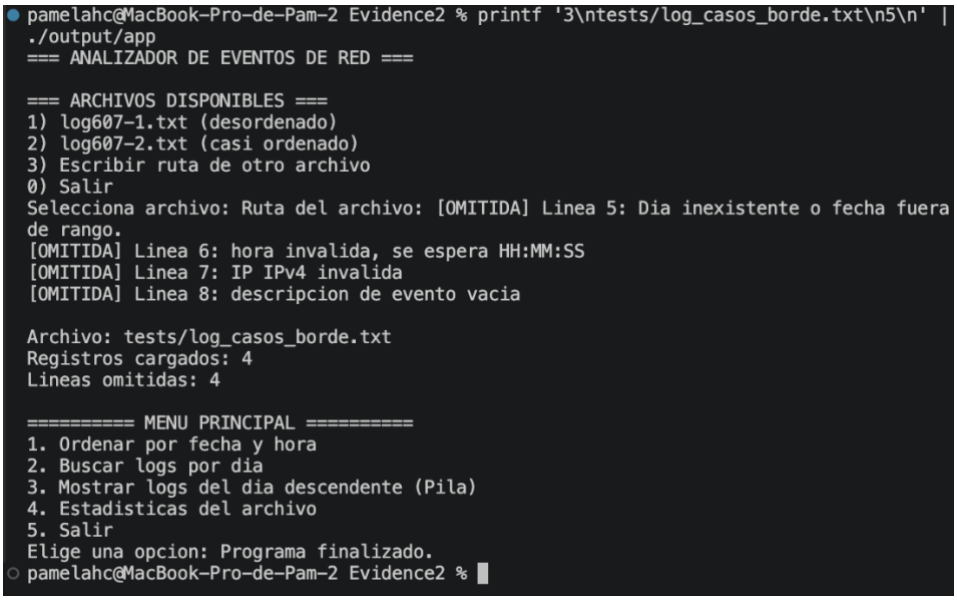


Figura 10. ***Archivo inexistente: mensaje y retorno al menu***

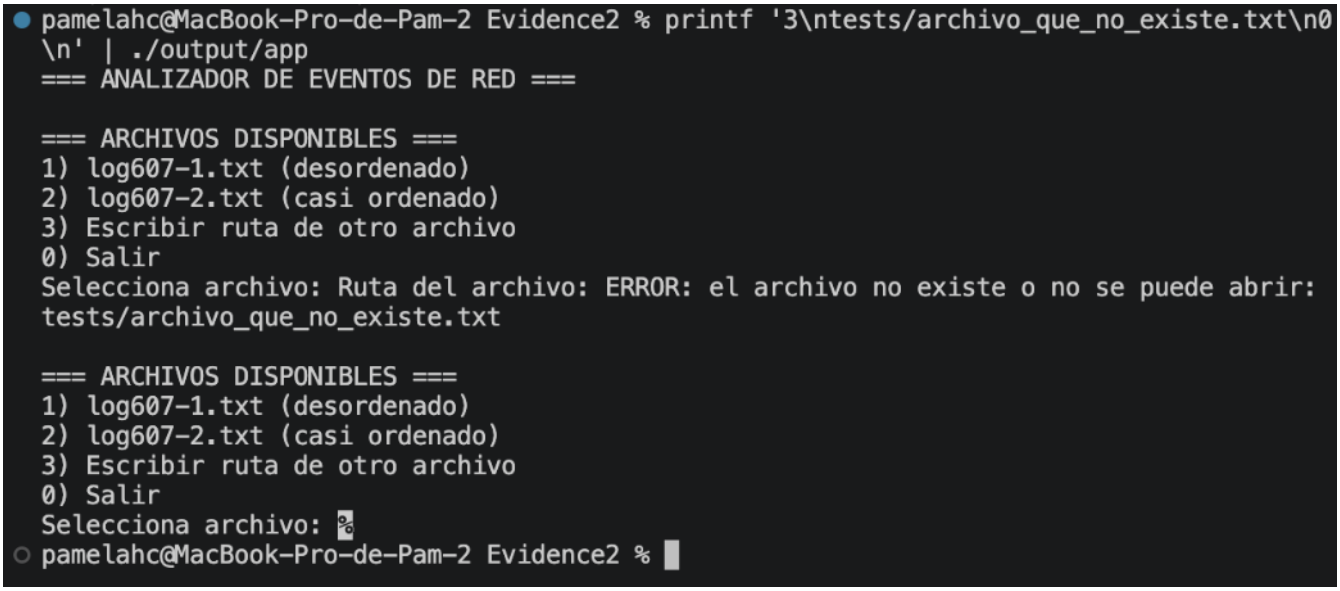


Figura 11. ***Busqueda bloqueada antes de ejecutar Merge Sort***

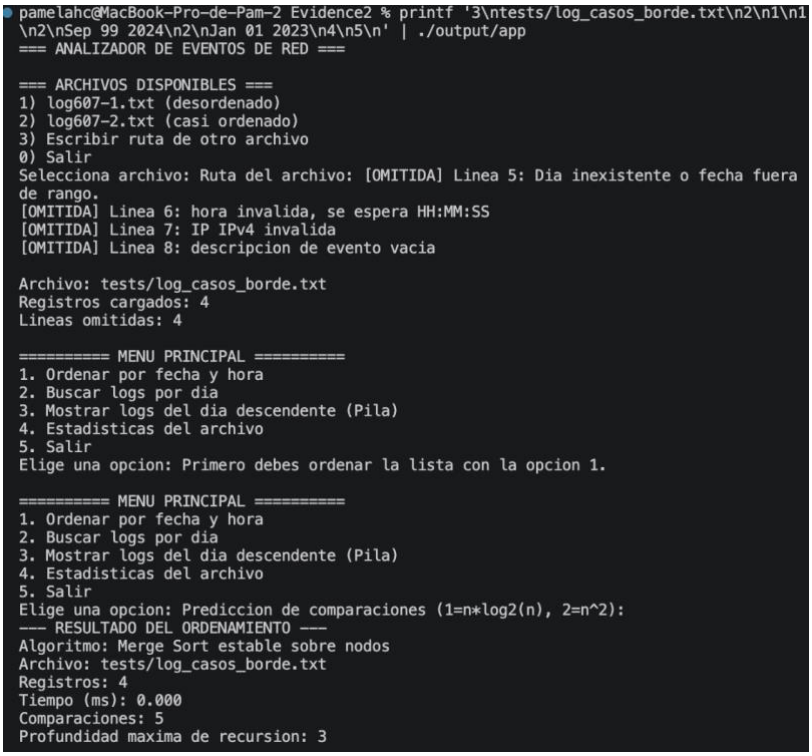


Figura 12. ***Fecha invalida y busqueda sin resultados***



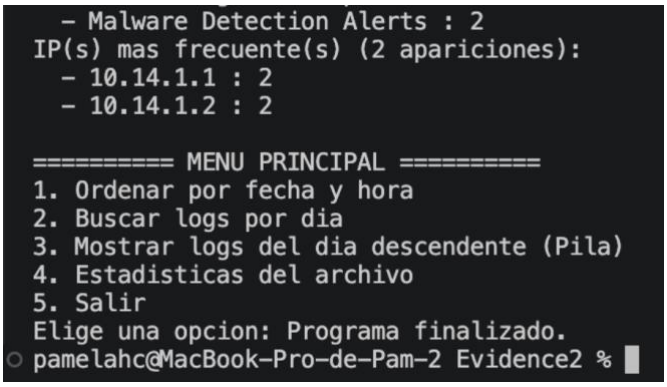
La fecha Sep 99 2024 se rechaza. Jan 01 2023 genera cero coincidencias sin interrumpir la ejecucion. El programa

muestra tambien estadisticas con empates

Empates y control de salida

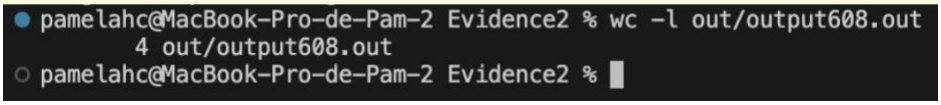
Se documentan los empates en IP y el estado temporal del archivo de salida tras las pruebas pequenas.

Figura 13. ***Empate de direcciones IP y fin de ejecucion***



Las IP 10.14.1.1 y 10.14.1.2 empatan con dos apariciones cada una.

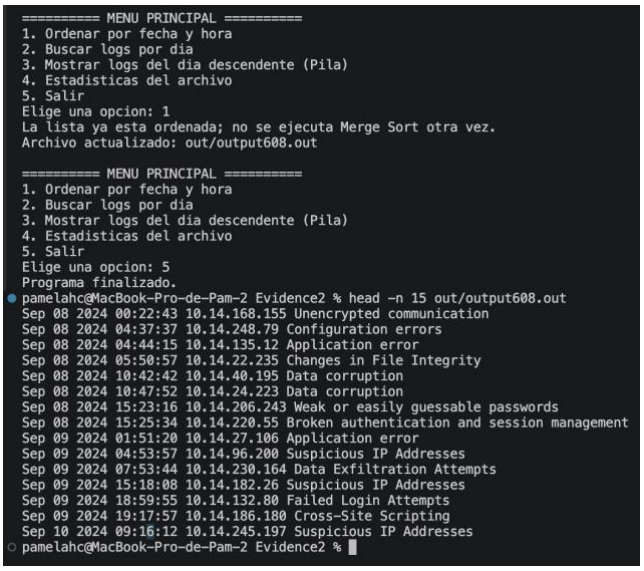
Figura 14. ***Control temporal: output608.out con cuatro registros***



7. Archivo generado

Tras repetir el proceso con los datos completos, se recuperó out/output608.out con 6,818 líneas en el orden cronológico esperado. El archivo queda incluido en el repositorio (se agregó con git add -f porque .gitignore ignora *.out).

Figura 15. ***Archivo de salida***



Mensaje de lista previamente ordenada y primeras quince líneas de out/output608.out, ordenadas cronológicamente. Fuente: captura de terminal proporcionada para la evidencia.

8. Historial de Git y evidencia del proceso

Commit	Cambio verificable
56dd70e	Estructuras, lectura y código principal
fbb4ae9	Pruebas Parte A, corrida de log607-2 y output608.out
fb22074	Casos de borde, archivo inexistente y repetición del segundo log

Repositorio: https://github.com/tecnologico-de-monterrey-oficial/C.MTY.TC1031.607.2613.A01771362

9. Uso de IA: indicaciones y aportaciones

La actividad distingue una Parte A que debía trabajarse en clase sin asistentes y una Parte B donde se documenta el apoyo recibido. Antes de entregar, verificar y declarar con precisión qué trabajo fue realizado personalmente, qué código tuvo ayuda de IA y qué parte fue revisada y modificada.

Durante el desarrollo de esta actividad utilicé herramientas de inteligencia artificial como apoyo para resolver dudas, organizar el proyecto, revisar algunas partes de la programación y documentar las pruebas realizadas. La IA me ayudó principalmente a identificar errores, comprender los procedimientos necesarios para trabajar con Git y GitHub, y estructurar las evidencias solicitadas.

Prompts reales que sí aparecen en la conversación

Prompt 1:

"¿Cómo puedo comprobar el funcionamiento de las estructuras de datos lineales en C++, considerando operaciones con listas vacías, inserciones, eliminaciones y recorridos?"

Aportación: La IA proporcionó orientación sobre las pruebas necesarias para verificar las operaciones de las estructuras. Esto permitió revisar los resultados de la lista ligada, la Pila, la Fila y la lista doblemente ligada.

Prompt 2:

"¿Qué aspectos debo revisar al comparar el rendimiento de Merge Sort con un archivo desordenado y otro casi ordenado?"

Aportación: La IA ayudó a interpretar las mediciones de tiempo, número de comparaciones y profundidad máxima de recursión. Esto facilitó la comparación de resultados y su explicación en el reporte.

Prompt 3:

"¿Qué casos de prueba adicionales puedo utilizar para comprobar que mi programa maneja correctamente entradas inválidas y situaciones especiales?"

Aportación: La IA sugirió verificar casos como fechas incorrectas, direcciones IP inválidas, archivos inexistentes y registros con fechas repetidas. Las pruebas se ejecutaron y sus resultados se documentaron mediante capturas de pantalla.

Ejemplo concreto de un problema detectado y corregido

Uno de los errores que identifiqué al utilizar IA ocurrió cuando se sugirió ejecutar el archivo ./ejecutar_mac.sh para compilar el programa. Sin embargo, al revisar mi proyecto en Visual Studio Code, observé que ese archivo no existía.

Para solucionar el problema, utilicé directamente el compilador g++ desde la terminal, indicando las rutas de los archivos fuente y las librerías necesarias. De esta manera logré compilar y ejecutar correctamente tanto el programa principal como las pruebas de las estructuras de datos.

Esta experiencia me permitió comprender que las recomendaciones generadas por IA no siempre coinciden con la organización real de un proyecto. Por ello, es importante comprobar las instrucciones, identificar posibles errores y adaptar las soluciones antes de utilizarlas.

Política de IA para README y declaración de autoría

El desarrollo de esta actividad combinó trabajo personal con apoyo parcial de inteligencia artificial. La IA fue utilizada como herramienta de consulta, asistencia en programación, solución de dudas, organización de pruebas y apoyo en la documentación.

Por mi parte, ejecuté las pruebas desde mi computadora, comprobé los resultados del programa, realicé las capturas de pantalla y administré los commits y pushes de GitHub.

Finalmente, comprendí que la IA puede facilitar el proceso de aprendizaje y desarrollo, pero no sustituye la necesidad de comprender el código, comprobar su funcionamiento y tomar decisiones propias.

10. Reflexión personal

En esta actividad comprendí que seleccionar una estructura de datos no depende solo de si puede almacenar información, sino de cómo recibe, conserva y presenta los datos. La Fila fue adecuada para procesar cada línea en el mismo orden en que se encontraba en el archivo. Al desencolar cada elemento, el programa podía validar su formato y conservar el orden de llegada de los registros válidos. Esta separación entre la entrada y el almacenamiento final también permitió manejar errores individuales sin detener todo el proceso.

La lista doblemente ligada fue el centro de la aplicación porque guardó los registros de manera dinámica y permitió aplicar Merge Sort mediante enlaces entre nodos. Elegir un Merge Sort estable ayudó a mantener el orden original de registros con exactamente la misma fecha y hora, propiedad importante para representar eventos que provienen de distintas fuentes. La Pila, aunque no era estrictamente necesaria porque una lista doble posee apuntadores prev y se puede recorrer hacia atrás, sirvió para practicar LIFO y cumplir el objetivo de reutilizar las estructuras implementadas. Su desventaja es que requiere apilar referencias durante la consulta; el recorrido inverso directo sería más sencillo y usaría menos memoria auxiliar.

Al revisar las mediciones observé que el archivo aleatorio produjo 78,330 comparaciones, mientras que el casi ordenado produjo 50,624, aun cuando ambos contenían 6,818 registros. La profundidad máxima de recursión fue 14 en ambos. Esta profundidad está relacionada con las llamadas activas de Merge Sort: cada división genera llamadas sobre fragmentos cada vez más pequeños hasta llegar a casos base. Por eso se relaciona con la memoria usada en la pila de llamadas y con la altura del proceso recursivo, no directamente con la cantidad de comparaciones realizada.

También entendí mejor por qué no puede aplicarse búsqueda binaria directamente sobre una lista ligada: no existe acceso aleatorio en tiempo constante al elemento de una posición. Llegar al elemento central requiere recorrer nodos, lo que elimina la ventaja habitual de la búsqueda binaria. Para hacer búsquedas binarias eficientes sería necesario construir un índice de posiciones o un arreglo de punteros a nodos, con almacenamiento auxiliar y costo de mantenimiento cuando la lista cambie.

En cuanto a la IA, me resultó útil para organizar una lista extensa de requisitos, interpretar mensajes de Git y proponer pruebas reproducibles. Sin embargo, observé que una recomendación puede referirse a archivos que no existen o a pasos que requieren ajustes al proyecto real. Por ello, fue importante comprobar cada sugerencia con el compilador, los resultados de ejecución y el repositorio. Durante el desarrollo de esta actividad, realicé parte del trabajo de programación y utilicé herramientas de inteligencia artificial como apoyo en otras etapas. La IA me ayudó a comprender algunos procedimientos, organizar las pruebas, interpretar resultados y resolver dudas relacionadas con Git y GitHub. También comprobé el funcionamiento de la aplicación mediante la compilación y ejecución de pruebas, revisando los resultados obtenidos, habría dedicado más tiempo a consultar documentación de Git y diseñar casos de prueba, pero habría seguido necesitando verificar que el código cumpliera las restricciones de la rúbrica.

11. Estructura de carpetas y reproducción

```
Evidences/Evidence2/
  README.md
  Structures.h
  data/log607-1.txt
  data/log607-2.txt
  include/{Node,LinkedList,Queue,Stack,DoublyLinkedList}.h
  src/main.cpp / src/tests.cpp
  tests/log_casos_borde.txt
  docs/pruebas_estructuras.txt
  docs/ejecucion_log607-2.txt
  docs/pruebas_casos_borde.txt
  docs/prueba_archivo_inexistente.txt
  out/output608.out
  docs/Evidencia2.pdf
```

```
cd Evidences/Evidence2
mkdir -p output out
g++ -std=c++17 -O2 -Wall -Wextra -pedantic -Iinclude src/main.cpp -o output/app
g++ -std=c++17 -O2 -Wall -Wextra -pedantic -Iinclude src/tests.cpp -o output/tests_estructuras
./output/tests_estructuras
./output/app
```

12. Video explicativo

Enlace Drive o YouTube no listado:

https://youtu.be/GkUFwjLrNLw

Grabación solicitada: 3-5 minutos, pantalla y voz, sin cortes. Presentar el uso de Fila, lista doble, Pila, Merge Sort estable y ejecución de ambos logs. Copiar el mismo enlace a README.md.